

Low-Rank Tensor Approximation for Image
Compression: A Comparative Study of
Decomposition Methods, Optimization Algorithms,
and Reconstruction Norms

Mathematical Analysis and Numerical Experiments on Norm-dependent Reconstruction
Behavior

May 21, 2026

Project Information	
Course	TP406 Geometry of Tensors and Applications
Instructor	Kangjin Han
Submission Date	May 21, 2026
Team Number	04
Contributors	김동현, 박사빈, 정범준, 조수호

This page intentionally left blank.

Revision History

The following table summarizes the major revisions and updates applied to this document.

Version	Date	Author	Description
0.1	2026-05-21	Beomjun Chung	Initial structure

Table 1: Revision history of the document

This page intentionally left blank.

Abstract

This report presents the design and implementation of an Attitude and Heading Reference System using the MPU-9250 Inertial Measurement Unit. The study covers the full pipeline from sensor characterization to state estimation, including stochastic noise modeling via Allan Variance and Power Spectral Density analysis, static calibration of the accelerometer, and Hard/Soft Iron correction for the magnetometer.

Three attitude estimation algorithms are designed and compared: a Complementary Filter, the Mahony and Madgwick filters, and an Extended Kalman Filter. The EKF is formulated with a quaternion-based state representation to avoid gimbal lock and provide globally stable attitude estimates. Algorithm performance is first validated through Python-based simulation under controlled motion profiles, then evaluated on hardware using an Arduino-based platform with the MPU-9250.

Keywords: Inertial Measurement Unit, Extended Kalman Filter, Attitude and Heading Reference System, Sensor Calibration, Allan Variance, Sensor Fusion

This page intentionally left blank.

Contents

1	Introduction	8
2	Mathematical Background	10
2.1	Tensor Notation	10
2.2	Tucker Decomposition	11
2.3	CP Decomposition	11
2.4	Higher-Order SVD (HOSVD)	12
2.5	Reconstruction Losses	12
3	Optimization Methods	14
3.1	Alternating Least Squares and HOOI	14
3.2	Iteratively Reweighted Least Squares (IRLS) for ℓ_1 -Tucker	15
3.3	Gradient-Based Optimization for Huber-Tucker	15
3.4	Comparison of Methods	16
4	Norm Analysis	18
4.1	Frobenius Norm: Statistical and Geometric Properties	18
4.2	ℓ_1 Norm: Robustness and Non-Smoothness	18
4.3	Huber Loss: Smooth Interpolation	19
4.4	Geometric Interpretation	19
4.5	Norm Comparison Under Structured Noise	20
5	Image Compression Demo	21
6	Discussion & Conclusion	22
	Appendix	24
	Appendix	25
A	Quaternion Algebra	25
B	EKF Jacobian Derivation	26
C	Code Structure	28
D	MPU-9250 Hardware Overview	30

This page intentionally left blank.

1. Introduction

Digital images are conventionally stored and processed as three-dimensional arrays of shape $H \times W \times C$, where H and W denote the spatial dimensions and C the number of color channels. Although matrix-based compression methods such as truncated Singular Value Decomposition (SVD) are well-established, they require either flattening the channel axis or processing each channel independently, thereby discarding inter-channel correlations that carry perceptually significant structure.

Tensor decomposition methods naturally preserve this multi-way structure. A *tensor* $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ generalizes matrices to N modes, and *low-rank tensor approximation* seeks a compact representation $\hat{\mathcal{X}}$ such that

$$\hat{\mathcal{X}} = \arg \min_{\hat{\mathcal{X}} \in \mathcal{M}_r} \mathcal{L}(\mathcal{X}, \hat{\mathcal{X}}), \quad (1.1)$$

where $\mathcal{M}_r$ denotes a suitable low-rank manifold and $\mathcal{L}$ is a reconstruction loss. The choice of $\mathcal{L}$ is consequential: the squared Frobenius norm $\|\cdot\|_F^2$ yields analytically tractable subproblems and is the standard baseline, yet it is sensitive to outliers and heavy-tailed noise. Alternative losses—the ℓ_1 norm and the Huber loss—offer robustness at the cost of algorithmic complexity, motivating a comparative study.

Research Questions

This project investigates the following questions:

1. **Decomposition structure.** How do Tucker decomposition and CP decomposition differ in their approximation quality, compression ratio, and computational cost for image tensors?
2. **Norm dependence.** How does the choice of reconstruction norm ($\|\cdot\|_F^2$, $\|\cdot\|_1$, Huber loss) affect the recovered factor matrices and the visual quality of the reconstructed image, particularly under structured noise?
3. **Optimization behavior.** How do Alternating Least Squares (ALS) / Higher-Order Orthogonal Iteration (HOOI) compare with gradient-based methods (Adam, L-BFGS) in terms of convergence speed, stability, and sensitivity to initialization?
4. **Rank–quality tradeoff.** Can the theoretical error bound derived from the multilinear singular value spectrum predict the empirically observed reconstruction error as a function of Tucker rank?

Contributions

The contributions of this work are as follows.

- We implement Tucker decomposition from scratch (HOSVD initialization and HOOI refinement) and verify correctness against TensorLy [4].
- We derive and implement an Iteratively Reweighted Least Squares (IRLS) algorithm for ℓ_1 -Tucker and compare it with the Frobenius baseline and Huber-loss gradient descent on noisy image tensors.
- We conduct controlled numerical experiments on rank truncation, norm sensitivity, convergence, and initialization, connecting each empirical result to its theoretical prediction.

- We demonstrate a lightweight image compression pipeline that reports PSNR and compression ratio as functions of Tucker rank, providing an interpretable application of the mathematical framework.

Paper Organization

The remainder of this report is organized as follows. Section 2 establishes the mathematical background, including tensor notation, Tucker and CP decomposition, and the norms used throughout. Section 3 derives the optimization algorithms: ALS/HOOI for the Frobenius objective and IRLS plus gradient-based methods for robust objectives. Section 4 analyzes the statistical and geometric implications of each norm choice. Section ?? presents numerical experiments that validate the theoretical predictions. Section ?? applies the framework to image compression. Section ?? interprets the results and discusses limitations, and Section ?? concludes.

2. Mathematical Background

This section establishes the notation and mathematical foundations used throughout the report. We introduce tensor algebra, the two decomposition families under study (Tucker and CP), the multilinear singular value decomposition (HOSVD), and the reconstruction losses that define each optimization problem.

2.1. Tensor Notation

A *tensor* $X \in I_1 \times I_2 \times \dots \times I_N$ is an N -way array. Its entry indexed by $(i_1, i_2, \dots, i_N)$ is written $x_{i_1 i_2 \dots i_N}$. We adopt the following notational convention throughout: calligraphic letters (X) for tensors, bold uppercase (U) for matrices, bold lowercase (u) for vectors, and italic lowercase (λ, r) for scalars.

Frobenius norm. The Frobenius norm of a tensor generalizes the matrix Frobenius norm entry-wise:

$$\|X\|_F = \sqrt{\sum_{i_1=1}^{I_1} \dots \sum_{i_N=1}^{I_N} x_{i_1 \dots i_N}^2} = \sqrt{\langle X, X \rangle}. \quad (2.1)$$

It satisfies *unitary invariance*: for any orthogonal matrices Q_n ,

$$X \times_1 Q_1 \times_2 Q_2 \dots \times_N Q_N = X. \quad (2.2)$$

This property is central to the correctness of HOSVD and HOOI (Section 2.4).

Mode- n unfolding (matricization). The *mode- n unfolding* $X_n \in I_n \times \prod_{k \neq n} I_k$ rearranges the entries of X such that the mode- n index maps to rows and all remaining indices (in lexicographic order) map to columns [3]. Concretely,

$$[X_n]_{i_n, j} = x_{i_1 i_2 \dots i_N}, \quad j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m. \quad (2.3)$$

Mode- n product. The *mode- n product* of $X \in I_1 \times \dots \times I_N$ with $U \in J \times I_n$ is the tensor $Y = X \times_n U \in I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N$ defined entry-wise as

$$y_{i_1 \dots i_{n-1} j i_{n+1} \dots i_N} = \sum_{i_n=1}^{I_n} x_{i_1 \dots i_N} u_{j i_n}. \quad (2.4)$$

In matricized form this becomes

$$(X \times_n U)_{(n)} = U X_n, \quad (2.5)$$

which reduces the mode- n product to ordinary matrix multiplication and underpins the ALS update rules derived in Section 3.1.

Commutativity of mode products. For $m \neq n$,

$$X \times_m A \times_n B = X \times_n B \times_m A. \quad (2.6)$$

This commutativity is used repeatedly in the derivation of ALS subproblems.

2.2. Tucker Decomposition

[Tucker decomposition] A rank- $(r_1, \dots, r_N)$ Tucker approximation of $X \in I_1 \times \dots \times I_N$ is

$$\hat{X} = G \times_1 U^{(1)} \times_2 U^{(2)} \dots \times_N U^{(N)}, \quad (2.7)$$

where $G \in r_1 \times \dots \times r_N$ is the *core tensor* and each factor matrix $U^{(n)} \in I_n \times r_n$ has orthonormal columns, i.e., $U^{(n)\top} U^{(n)} = I_{r_n}$.

Mode- n unfolding of $\hat{X}$ yields

$$\hat{X}_{(n)} = U^{(n)} G_{(n)} \left(U^{(N)} \dots U^{(n+1)} U^{(n-1)} \dots U^{(1)} \right)^\top, \quad (2.8)$$

where $\times$ denotes the Kronecker product. Equation (2.8) exposes the least-squares structure exploited by ALS.

Parameter count and compression ratio. The Tucker format requires

$$P_{\text{Tucker}} = \prod_{n=1}^N r_n + \sum_{n=1}^N I_n r_n \quad (2.9)$$

parameters. The compression ratio relative to the dense tensor is

$$\rho = \frac{\prod_{n=1}^N I_n}{P_{\text{Tucker}}}. \quad (2.10)$$

For a three-channel image of size $H \times W \times 3$ with Tucker rank $(r, r, 3)$, this gives $\rho = HW \cdot 3 / (3r^2 + Hr + Wr + 3r)$, which grows roughly as HW/r^2 for large images.

Relationship to truncated SVD. For $N = 2$, Tucker reduces to the truncated SVD: $\hat{X} = U \Sigma_r V^\top$, with $G = \Sigma_r$ and $U^{(1)} = U$, $U^{(2)} = V$. Tucker thus provides the natural multilinear generalization of low-rank matrix approximation.

2.3. CP Decomposition

[CP decomposition] A rank- R CANDECOMP/PARAFAC (CP) approximation of X is

$$\hat{X} = \sum_{r=1}^R \lambda_r a_r^{(1)} \circ a_r^{(2)} \circ \dots \circ a_r^{(N)}, \quad (2.11)$$

where $\circ$ denotes the outer product, $\|a_r^{(n)}\|_2 = 1$, and $\lambda_r \geq 0$.

CP can be viewed as Tucker with a *superdiagonal* core of size $R \times \dots \times R$, i.e., the case $r_1 = \dots = r_N = R$ with $g_{i_1 \dots i_N} = \lambda_{i_1}$ only when $i_1 = i_2 = \dots = i_N$ and zero otherwise. The CP format stores $R(\sum_{n=1}^N I_n + 1)$ parameters, fewer than Tucker for the same R , but determining the exact CP rank is NP-hard in general [3].

[CP degeneracy] Unlike Tucker, the best rank- R CP approximation of a tensor of order $N \geq 3$ may not exist: the infimum of the objective is not always attained [3]. Tucker decomposition avoids this pathology because orthonormal factor matrices form a compact set.

2.4. Higher-Order SVD (HOSVD)

[HOSVD [1]] The *Higher-Order SVD* of X computes each factor matrix independently:

$$U^{(n)} := \text{leading } r_n \text{ left singular vectors of } X_n, \quad n = 1, \dots, N, \quad (2.12)$$

and sets the core tensor to

$$G = X \times_1 U^{(1)\top} \times_2 U^{(2)\top} \dots \times_N U^{(N)\top}. \quad (2.13)$$

HOSVD is computed in closed form and provides a deterministic initialization. However, it is *not* the globally optimal Tucker approximation because the mode- n factor matrices are optimized independently rather than jointly.

[HOSVD error bound [1]] Let $\hat{X}_{\text{HOSVD}}$ denote the rank- $(r_1, \dots, r_N)$ HOSVD approximation of X . Then

$$\|X - \hat{X}_{\text{HOSVD}}\|_F^2 \leq \sum_{n=1}^N \sum_{i=r_n+1}^{I_n} \sigma_i^{(n)2}, \quad (2.14)$$

where $\sigma_i^{(n)}$ denotes the i -th singular value of X_n .

Equation (2.14) is used in Section ?? to benchmark the rank sweep experiment: the left-hand side is measured empirically and compared with the right-hand side computed from the singular value spectra.

2.5. Reconstruction Losses

The general approximation problem is

$$\min_{G, \{U^{(n)}\}} \mathcal{L}(X, \hat{X}), \quad (2.15)$$

where the choice of $\mathcal{L}$ determines both the statistical interpretation of the solution and the algorithmic complexity of the optimization. We study three loss functions.

Squared Frobenius loss.

$$\mathcal{L}_F(X, \hat{X}) = \|X - \hat{X}\|_F^2 = \sum_{i_1, \dots, i_N} (x_{i_1 \dots i_N} - \hat{x}_{i_1 \dots i_N})^2. \quad (2.16)$$

This loss is smooth and convex in each factor separately, enabling closed-form ALS updates. When the noise is i.i.d. Gaussian, the Frobenius minimizer is the maximum likelihood estimator. Its sensitivity to large residuals, however, makes it non-robust to outliers.

ℓ_1 loss.

$$\mathcal{L}_1(X, \hat{X}) = \|X - \hat{X}\|_1 = \sum_{i_1, \dots, i_N} |x_{i_1 \dots i_N} - \hat{x}_{i_1 \dots i_N}|. \quad (2.17)$$

Minimizing $\mathcal{L}_1$ corresponds to maximum likelihood estimation under a Laplace noise model and yields a positive breakdown point, making it robust to sparse outliers. The loss is convex but *non-smooth* at zero; its subdifferential is $\partial|\cdot| = (\cdot)$, which precludes direct use of standard least-squares solvers.

Huber loss. The Huber loss [2] interpolates smoothly between ℓ_2 (near zero) and ℓ_1 (in the tails):

$$\rho_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| \leq \delta, \\ \delta\left(|r| - \frac{\delta}{2}\right) & \text{if } |r| > \delta, \end{cases} \quad (2.18)$$

with threshold $\delta > 0$. The aggregate loss is $\mathcal{L}_H = \sum_{i_1, \dots, i_N} \rho_\delta(x_{i_1 \dots i_N} - \hat{x}_{i_1 \dots i_N})$. Being everywhere differentiable, $\mathcal{L}_H$ admits gradient-based optimization without subgradient complications.

Summary. Table 2 summarizes the three losses with respect to statistical interpretation, smoothness, and optimization strategy.

Table 2: Comparison of reconstruction losses.

Loss	Noise model	Smoothness	Optimization
$\mathcal{L}_F$ (Frobenius)	Gaussian	C^∞	ALS (closed-form)
$\mathcal{L}_1$ (ℓ_1)	Laplace	non-smooth at 0	IRLS / subgradient
$\mathcal{L}_H$ (Huber)	Gaussian + outliers	C^1	Gradient descent

3. Optimization Methods

The Tucker approximation problem (2.15) is *jointly non-convex* in the factor matrices $\{U^{(n)}\}$ and the core tensor G . We study three algorithmic families that address this non-convexity in different ways: Alternating Least Squares (ALS) with its higher-order refinement HOOI for the Frobenius objective, Iteratively Reweighted Least Squares (IRLS) for the ℓ_1 objective, and gradient-based methods for the Huber objective.

3.1. Alternating Least Squares and HOOI

ALS principle. Although $\mathcal{L}_F$ is jointly non-convex in all factors simultaneously, it is *quadratic* — and hence convex — when all but one factor matrix are held fixed. ALS exploits this structure by cycling over the factors and solving each resulting least-squares problem exactly.

Mode- n subproblem. Fix all factor matrices except $U^{(n)}$ and define

$$Z^{(n)} := G_{(n)} \left(U^{(N)} \dots U^{(n+1)} U^{(n-1)} \dots U^{(1)} \right)^\top \in {}^{r_n \times \prod_{k \neq n} I_k}. \quad (3.1)$$

The objective reduces to

$$\min_{U^{(n)}} \|Xn - U^{(n)} Z^{(n)}\|_F^2, \quad (3.2)$$

whose closed-form solution is

$$U^{(n)} \leftarrow Xn Z^{(n)\top} \left(Z^{(n)} Z^{(n)\top} \right)^{-1}. \quad (3.3)$$

When the columns of $U^{(n)}$ are additionally constrained to be orthonormal — as in Tucker — the optimal solution is the matrix of leading r_n left singular vectors of $Xn Z^{(n)\top}$, making the matrix inverse unnecessary.

HOOI algorithm. Higher-Order Orthogonal Iteration (HOOI) [?] applies ALS with orthonormal constraints. At each iteration, the contracted tensor

$$Y^{(n)} = X \times_1 U^{(1)\top} \dots \times_{n-1} U^{(n-1)\top} \times_{n+1} U^{(n+1)\top} \dots \times_N U^{(N)\top} \quad (3.4)$$

is formed (omitting mode n), and the update is

$$U^{(n)} \leftarrow \text{leading-}r_n \text{ left singular vectors of } Y_{(n)}^{(n)}. \quad (3.5)$$

After all factor matrices have converged, the core is recovered as

$$G = X \times_1 U^{(1)\top} \times_2 U^{(2)\top} \dots \times_N U^{(N)\top}. \quad (3.6)$$

Algorithm 1 summarizes the full procedure.

Convergence. Each HOOI iteration decreases the Frobenius reconstruction error monotonically, because each mode- n update is an exact minimizer of the subproblem (3.2) given the other factors. The sequence $\{e_t\}$ is therefore non-increasing and bounded below by zero, guaranteeing convergence of the *objective value*. However, convergence of the *iterates* $\{U_t^{(n)}\}$ to a stationary point is not guaranteed in general due to non-convexity [?].

Computational cost. The dominant cost per HOOI iteration is N truncated SVDs of matrices of size $I_n \times \prod_{k \neq n} r_k$, giving a per-iteration complexity of $O\left(N \cdot I \cdot r^{N-1}\right)$ for a balanced tensor with $I_n = I$ and $r_n = r$.

Algorithm 1 HOOI for Tucker decomposition (Frobenius norm)

Tensor $X \in I_1 \times \dots \times I_N$, target rank $(r_1, \dots, r_N)$, tolerance $\varepsilon > 0$, max iterations T Core G , factor matrices $\{U^{(n)}\}$

Initialization via HOSVD

for $n \leftarrow 1$ N **do** $U^{(n)} \leftarrow$ leading- r_n left singular vectors of X_n Iterative refinement

for $t \leftarrow 1$ T **do**

for $n \leftarrow 1$ N **do** $Y \leftarrow X \times_{k \neq n} U^{(k)\top}$ $U^{(n)} \leftarrow$ leading- r_n left singular vectors of Y_n
 $G \leftarrow X \times_1 U^{(1)\top} \dots \times_N U^{(N)\top}$ $e_t \leftarrow X - G \times_1 U^{(1)} \dots \times_N U^{(N)}$ $|e_t - e_{t-1}| < \varepsilon$ **break** **return** $G, \{U^{(n)}\}$

3.2. Iteratively Reweighted Least Squares (IRLS) for ℓ_1 -Tucker

Motivation. Direct minimization of $\mathcal{L}_1$ via subgradient methods converges at rate $O(1/\sqrt{t})$, which is slow in practice. IRLS sidesteps non-smoothness by solving a sequence of *weighted* Frobenius problems, each of which admits a standard ALS step.

IRLS reweighting. Let $r_{i_1 \dots i_N} = x_{i_1 \dots i_N} - \hat{x}_{i_1 \dots i_N}$ be the current residual tensor. Define element-wise weights

$$w_{i_1 \dots i_N}^{(t)} = \frac{1}{|r_{i_1 \dots i_N}^{(t)}| + \varepsilon}, \quad \varepsilon > 0 \text{ (smoothing parameter)}, \quad (3.7)$$

and the *weighted* Frobenius problem

$$\min_{G, \{U^{(n)}\}} \sum_{i_1, \dots, i_N} w_{i_1 \dots i_N}^{(t)} (x_{i_1 \dots i_N} - \hat{x}_{i_1 \dots i_N})^2. \quad (3.8)$$

The connection to ℓ_1 follows from the identity

$$|r| = w \cdot r^2 \quad \text{when } w = 1/|r|, \quad (3.9)$$

which shows that minimizing (3.8) with fixed weights is a majorization step for $\mathcal{L}_1$.

Weighted ALS update. Let $W^{(t)}$ be the weight tensor and define the element-wise product $\tilde{X} = W^{(t)} \odot X$. The weighted mode- n subproblem is

$$\min_{U^{(n)}} \|\tilde{X}_{(n)} - U^{(n)} \tilde{Z}^{(n)}\|_F^2, \quad (3.10)$$

which is solved by (3.3) applied to the weighted unfolding $\tilde{X}_{(n)}$.

Algorithm 2 presents the full IRLS procedure.

Choice of ε . The parameter ε prevents division by zero and controls the approximation of $|\cdot|$ near the origin. Smaller ε gives a closer approximation to the true ℓ_1 loss but may cause numerical instability. A practical schedule starts with $\varepsilon = 10^{-1}$ and halves it every few outer iterations.

3.3. Gradient-Based Optimization for Huber-Tucker

Motivation. The Huber loss $\mathcal{L}_H$ is everywhere differentiable, enabling direct application of first- and second-order gradient methods. We use this objective to compare gradient-based optimization with ALS/IRLS in terms of convergence speed and final reconstruction quality.

Algorithm 2 IRLS for ℓ_1 -Tucker decomposition

Tensor X , rank $(r_1, \dots, r_N)$, outer iterations T , inner ALS steps K , smoothing $\varepsilon > 0$ Core G , factor matrices $\{U^{(n)}\}$

$G, \{U^{(n)}\} \leftarrow \text{HOOI}(X, \mathbf{r})$ *warm start from Frobenius solution

for $t \leftarrow 1$ **to** T **do** $\hat{X} \leftarrow G \times_1 U^{(1)} \dots \times_N U^{(N)}$ $R \leftarrow X - \hat{X}$ *residual $W \leftarrow 1 / (|R| + \varepsilon)$ *element-wise weights (3.7)

for $k \leftarrow 1$ **to** K **do** One pass of weighted ALS

for $n \leftarrow 1$ **to** N **do** $U^{(n)} \leftarrow$ weighted ALS update (3.10) $G \leftarrow X \times_1 U^{(1)\top} \dots \times_N U^{(N)\top}$
return $G, \{U^{(n)}\}$

Gradient derivation. Let $R = X - \hat{X}$ be the residual and define the element-wise Huber pseudo-residual

$$\psi_\delta(r) = \frac{\partial \rho_\delta(r)}{\partial r} = \begin{cases} r & |r| \leq \delta, \\ \delta(r) & |r| > \delta. \end{cases} \quad (3.11)$$

The gradient of $\mathcal{L}_H$ with respect to $\hat{X}$ is

$$\frac{\partial \mathcal{L}_H}{\partial \hat{X}} = -\Psi_\delta(R), \quad (3.12)$$

where Ψ_δ is applied element-wise. By the chain rule and the mode- n product structure,

$$\frac{\partial \mathcal{L}_H}{\partial U^{(n)}} = -\Psi_\delta(R)_{(n)} \left(U^{(N)} \dots \widehat{U^{(n)}} \dots U^{(1)} \right) G_{(n)}^\top, \quad (3.13)$$

where $\widehat{}$ denotes omission of the n -th factor. The gradient with respect to the core is

$$\frac{\partial \mathcal{L}_H}{\partial G_{(n)}} = -U^{(n)\top} \Psi_\delta(R)_{(n)} \left(U^{(N)} \dots U^{(1)} \right). \quad (3.14)$$

Optimization algorithm. We employ Adam [?] with parameters $\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. All factor matrices are initialized from HOSVD to ensure a fair comparison with HOOI. The Huber threshold is set to $\delta = 0.1 \cdot \max |R_0|$, where R_0 is the initial residual, so that δ adapts to the scale of each tensor.

Implementation note. Equations (3.13)–(3.14) can be evaluated without autograd by computing mode- n products explicitly, which avoids the memory overhead of storing a computation graph over the full tensor. For moderate-sized tensors (up to $256 \times 256 \times 3$), this manual gradient implementation is preferred.

3.4. Comparison of Methods

Table 3 summarizes the three optimization methods along the axes most relevant to this project.

Table 3: Comparison of optimization methods for Tucker approximation.

Method	Objective	Convergence	Per-iter. cost	Hyperparams
ALS / HOOI	$\mathcal{L}_F$	Monotone $\downarrow$, local	$O(NIr^{N-1})$	none
IRLS	$\mathcal{L}_1$	Monotone (outer), local	$O(NIr^{N-1})$	ε, K
Adam (gradient)	$\mathcal{L}_H$	Non-monotone, local	$O(NIr^{N-1})$	$\alpha, \delta, \beta_{1,2}$

Non-convexity and local minima. All three methods are susceptible to local minima because the feasible set $\mathcal{M}_r$ is non-convex. To assess sensitivity to initialization, Section ?? repeats each experiment with five random orthogonal initializations and reports the distribution of final reconstruction errors.

Numerical stability of ALS. The normal equation in (3.3) requires inverting $Z^{(n)}Z^{(n)\top}$. When this matrix is ill-conditioned — which occurs when factor matrices are nearly linearly dependent — a Tikhonov regularization term μI is added:

$$U^{(n)} \leftarrow XnZ^{(n)\top} \left(Z^{(n)}Z^{(n)\top} + \mu I \right)^{-1}, \quad \mu = 10^{-8} \quad (3.15)$$

In our experiments, the orthonormal constraint in HOOI makes this regularization unnecessary.

4. Norm Analysis

The choice of reconstruction loss $\mathcal{L}$ is not merely a computational convenience: it encodes an explicit assumption about the noise distribution and determines which errors the approximation prioritises minimising. This section analyses the three losses introduced in Section 2.5 from statistical, geometric, and numerical perspectives, and derives the implications for Tucker reconstruction under each norm.

4.1. Frobenius Norm: Statistical and Geometric Properties

Maximum likelihood interpretation. Suppose the observed tensor is $X = \hat{X} + E$, where $e_{i_1 \dots i_N} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. The log-likelihood is

$$\log p(X | \hat{X}) = -\frac{1}{2\sigma^2} X - \hat{X}^2 + \text{const}, \quad (4.1)$$

so the maximum likelihood estimator of $\hat{X}$ coincides exactly with the Frobenius minimiser. Under this noise model, HOOI is optimal.

Unitary invariance and the ALS simplification. Unitary invariance (2.2) implies that when the factor matrices are orthonormal, the objective decomposes as

$$X - \hat{X}^2 = X^2 - G^2, \quad (4.2)$$

so *minimising the reconstruction error is equivalent to maximising the energy captured by the core tensor*. This is the variational characterisation exploited by HOSVD and HOOI.

Sensitivity to outliers. The squared loss assigns weight proportional to $|r|$ to each residual $r = x - \hat{x}$, meaning that a single large outlier can dominate the objective and distort all factor matrices. Formally, the *influence function* of the Frobenius estimator is unbounded [?]:

$$\text{IF}(r; \mathcal{L}_F) = r, \quad (4.3)$$

confirming zero robustness to gross errors.

4.2. ℓ_1 Norm: Robustness and Non-Smoothness

Maximum likelihood interpretation. If $e_{i_1 \dots i_N} \stackrel{\text{i.i.d.}}{\sim} \text{Laplace}(0, b)$, the log-likelihood is

$$\log p(X | \hat{X}) = -\frac{1}{b} X - \hat{X} + \text{const}, \quad (4.4)$$

making the ℓ_1 minimiser the maximum likelihood estimator under Laplace noise. The heavier tails of the Laplace distribution relative to the Gaussian reflect the outlier tolerance built into $\mathcal{L}_1$.

Breakdown point. The *breakdown point* ε^* of an estimator is the fraction of arbitrarily corrupted observations it can tolerate before becoming arbitrarily biased. The Frobenius estimator has $\varepsilon^* = 0$ (a single outlier suffices to push the estimate arbitrarily far), whereas the ℓ_1 estimator achieves $\varepsilon^* = 0.5$ for location problems [?]. In the Tucker setting, this translates to robustness against sparse corruption of up to half the tensor entries.

Subdifferential and optimality condition. Because $\mathcal{L}_1$ is non-smooth, classical gradient conditions do not apply. The subdifferential of $|r|$ at r is

$$\partial|r| = \begin{cases} \{(r)\} & r \neq 0, \\ [-1, 1] & r = 0, \end{cases} \quad (4.5)$$

and the optimality condition for (2.15) with $\mathcal{L} = \mathcal{L}_1$ requires that zero lie in the subdifferential of the objective with respect to each factor. This is the condition that IRLS (Section 3.2) seeks to satisfy at convergence.

Influence function. The influence function of the ℓ_1 estimator is bounded:

$$\text{IF}(r; \mathcal{L}_1) = (r), \quad (4.6)$$

confirming that a single outlier cannot push the estimate beyond a finite neighbourhood of the truth.

4.3. Huber Loss: Smooth Interpolation

Definition and properties. The Huber loss ρ_δ (2.18) satisfies:

- $\rho_\delta \in C^1()$: continuously differentiable everywhere, enabling gradient descent without sub-gradient complications.
- $\rho_\delta(r) = \frac{1}{2}r^2$ for $|r| \leq \delta$: quadratic near zero, consistent with Gaussian small errors.
- $\rho_\delta(r) \sim \delta|r|$ for $|r| \gg \delta$: linear in the tails, inheriting the robustness of ℓ_1 .
- $\rho_\delta \rightarrow \frac{1}{2}|\cdot|^2$ as $\delta \rightarrow \infty$ and $\rho_\delta \rightarrow \delta|\cdot|$ as $\delta \rightarrow 0$: Huber interpolates between pure Frobenius and pure ℓ_1 .

Influence function.

$$\text{IF}(r; \mathcal{L}_H) = \psi_\delta(r) = \begin{cases} r & |r| \leq \delta, \\ \delta \text{sgn}(r) & |r| > \delta, \end{cases} \quad (4.7)$$

which is bounded (unlike Frobenius) yet smooth (unlike ℓ_1).

Threshold selection. The parameter δ controls the boundary between the quadratic and linear regimes. A data-adaptive choice is

$$\delta^* = c \cdot \widehat{\text{MAD}}(R), \quad \widehat{\text{MAD}} = \text{median}(|r_{i_1 \dots i_N}|), \quad (4.8)$$

where $c \approx 1.345$ is the standard efficiency constant for Gaussian noise [2]. In practice we use $\delta = 0.1 \cdot \max |R_0|$ (Section 3.3) as a scale-adaptive initialisation.

4.4. Geometric Interpretation

Each norm induces a different geometry on the residual space $I_1 \times \dots \times I_N$.

- **Frobenius** (ℓ_2): unit ball is a hypersphere. The projection onto the low-rank manifold $\mathcal{M}_r$ is the Euclidean nearest point, computed exactly by HOSVD/HOOI.
- ℓ_1 : unit ball is a cross-polytope. The solution places large residuals at the vertices of the polytope, concentrating error on a small number of entries (sparse residual structure).

- **Huber:** unit ball interpolates between the two. The transition radius δ determines how quickly the ball transitions from a hypersphere centre to a polytope surface.

This geometric picture predicts a key experimental observation: under sparse corruption, $\mathcal{L}_1$ -Tucker concentrates its residual on the corrupted entries, leaving the clean entries nearly exactly reconstructed, whereas $\mathcal{L}_F$ -Tucker spreads the residual evenly across all entries. Section ?? verifies this prediction via error maps.

4.5. Norm Comparison Under Structured Noise

Let the observed tensor be $X = X^* + S + G$, where X^* is the clean low-rank signal, S is a sparse corruption tensor with $\|S\|_0 = s$ nonzeros, and G is dense Gaussian noise with variance σ^2 .

Table 4 summarises the expected behaviour of each estimator under this model.

Table 4: Expected reconstruction behaviour under structured noise.

Loss	Noise type		
	Dense Gaussian	Sparse outliers	Both
$\mathcal{L}_F$	Optimal (MLE)	Poor	Poor
$\mathcal{L}_1$	Suboptimal	Robust ($\epsilon^*=0.5$)	Moderate
$\mathcal{L}_H$	Near-optimal	Robust (δ -dependent)	Good

The key prediction is that the relative advantage of $\mathcal{L}_1$ and $\mathcal{L}_H$ over $\mathcal{L}_F$ should grow monotonically with the fraction of corrupted entries $s/\prod_n I_n$. This is tested directly in the noise robustness experiment (Section ??).

5. Image Compression Demo

6. Discussion & Conclusion

- Summary of findings - Implications for image compression - Future work and open questions

This page intentionally left blank.

Appendix

Supplementary Materials

Appendix

A. Quaternion Algebra

본 부록에서는 쿼터니언의 기본 대수 연산과 성질을 정리한다. 4장 및 5장의 수식 유도에 필요한 기초 내용을 포함한다.

A.1 기본 정의

쿼터니언 $\mathbf{q} \in \mathbb{H}$ 는 실수 스칼라 성분 q_0 와 허수 벡터 성분 $\mathbf{q}_v = [q_1, q_2, q_3]^\top$ 으로 구성된다:

$$\mathbf{q} = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} \quad (\text{A.1})$$

허수 단위 사이의 곱셈 규칙:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1 \quad (\text{A.2})$$

$$\mathbf{ij} = \mathbf{k}, \quad \mathbf{jk} = \mathbf{i}, \quad \mathbf{ki} = \mathbf{j}, \quad \mathbf{ji} = -\mathbf{k}, \quad \mathbf{kj} = -\mathbf{i}, \quad \mathbf{ik} = -\mathbf{j} \quad (\text{A.3})$$

A.2 기본 연산

노름 (Norm)

$$\|\mathbf{q}\| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} \quad (\text{A.4})$$

단위 쿼터니언: $\|\mathbf{q}\| = 1$

켈레 (Conjugate)

$$\mathbf{q}^* = \begin{bmatrix} q_0 \\ -\mathbf{q}_v \end{bmatrix} = \begin{bmatrix} q_0 \\ -q_1 \\ -q_2 \\ -q_3 \end{bmatrix} \quad (\text{A.5})$$

단위 쿼터니언의 역원: $\mathbf{q}^{-1} = \mathbf{q}^*$

쿼터니언 곱 (Hamilton Product) 두 쿼터니언 $\mathbf{p} = [p_0, \mathbf{p}_v]^\top$, $\mathbf{q} = [q_0, \mathbf{q}_v]^\top$ 의 곱:

$$\mathbf{p} \otimes \mathbf{q} = \begin{bmatrix} p_0q_0 - \mathbf{p}_v^\top \mathbf{q}_v \\ p_0\mathbf{q}_v + q_0\mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v \end{bmatrix} \quad (\text{A.6})$$

쿼터니언 곱은 일반적으로 교환 법칙이 성립하지 않는다: $\mathbf{p} \otimes \mathbf{q} \neq \mathbf{q} \otimes \mathbf{p}$

행렬 표현 좌측 곱 행렬 $\Xi(\mathbf{p}) \in \mathbb{R}^{4 \times 4}$:

$$\mathbf{p} \otimes \mathbf{q} = \Xi(\mathbf{p})\mathbf{q} = \begin{bmatrix} p_0 & -p_1 & -p_2 & -p_3 \\ p_1 & p_0 & -p_3 & p_2 \\ p_2 & p_3 & p_0 & -p_1 \\ p_3 & -p_2 & p_1 & p_0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} \quad (\text{A.7})$$

우측 곱 행렬 $\Psi(\mathbf{q}) \in \mathbb{R}^{4 \times 4}$:

$$\mathbf{p} \otimes \mathbf{q} = \Psi(\mathbf{q})\mathbf{p} = \begin{bmatrix} q_0 & -q_1 & -q_2 & -q_3 \\ q_1 & q_0 & q_3 & -q_2 \\ q_2 & -q_3 & q_0 & q_1 \\ q_3 & q_2 & -q_1 & q_0 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix} \quad (\text{A.8})$$

A.3 회전 표현

회전 축 $\hat{\mathbf{u}} \in \mathbb{R}^3$ ($\|\hat{\mathbf{u}}\| = 1$)에 대한 각도 α 회전의 단위 쿼터니언:

$$\mathbf{q} = \begin{bmatrix} \cos(\alpha/2) \\ \hat{\mathbf{u}} \sin(\alpha/2) \end{bmatrix} \quad (\text{A.9})$$

벡터 $\mathbf{v} \in \mathbb{R}^3$ 의 쿼터니언 회전:

$$\begin{bmatrix} 0 \\ \mathbf{v}' \end{bmatrix} = \mathbf{q} \otimes \begin{bmatrix} 0 \\ \mathbf{v} \end{bmatrix} \otimes \mathbf{q}^* \quad (\text{A.10})$$

쿼터니언-DCM 변환:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix} \quad (\text{A.11})$$

A.4 쿼터니언 보간 (SLERP)

두 단위 쿼터니언 $\mathbf{q}_0, \mathbf{q}_1$ 사이의 구면 선형 보간(Spherical Linear Interpolation, SLERP):

$$\text{Slerp}(\mathbf{q}_0, \mathbf{q}_1; t) = \mathbf{q}_0 (\mathbf{q}_0^* \otimes \mathbf{q}_1)^t, \quad t \in [0, 1] \quad (\text{A.12})$$

닫힌 형태:

$$\text{Slerp}(\mathbf{q}_0, \mathbf{q}_1; t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{q}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{q}_1, \quad \cos \Omega = \mathbf{q}_0^\top \mathbf{q}_1 \quad (\text{A.13})$$

B. EKF Jacobian Derivation

본 부록에서는 5장 EKF 설계에 사용된 Jacobian 행렬의 전체 유도 과정을 기술한다.

B.1 상태 방정식 Jacobian $\mathbf{F}_c$

상태 벡터 $\mathbf{x} = [\mathbf{q}^\top, \mathbf{b}_g^\top]^\top \in \mathbb{R}^7$ 에 대한 연속시간 상태 방정식:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \tilde{\boldsymbol{\omega}}) = \begin{bmatrix} \frac{1}{2} \boldsymbol{\Omega}(\tilde{\boldsymbol{\omega}} - \mathbf{b}_g) \mathbf{q} \\ \mathbf{0}_3 \end{bmatrix} \quad (\text{B.1})$$

$\partial \dot{\mathbf{q}} / \partial \mathbf{q}$ 계산 $\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\tilde{\boldsymbol{\omega}}) \mathbf{q}$ 이므로:

$$\frac{\partial \dot{\mathbf{q}}}{\partial \mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\tilde{\boldsymbol{\omega}}) = \frac{1}{2} \begin{bmatrix} 0 & -\hat{\omega}_x & -\hat{\omega}_y & -\hat{\omega}_z \\ \hat{\omega}_x & 0 & \hat{\omega}_z & -\hat{\omega}_y \\ \hat{\omega}_y & -\hat{\omega}_z & 0 & \hat{\omega}_x \\ \hat{\omega}_z & \hat{\omega}_y & -\hat{\omega}_x & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (\text{B.2})$$

여기서 $\hat{\boldsymbol{\omega}} = \tilde{\boldsymbol{\omega}} - \hat{\mathbf{b}}_g$ 이다.

$\partial \dot{\mathbf{q}} / \partial \mathbf{b}_g$ 계산 $\dot{\mathbf{q}} = \frac{1}{2} \Xi(\mathbf{q}) \begin{bmatrix} 0 \\ \hat{\omega} \end{bmatrix}$ 이고, $\hat{\omega} = \tilde{\omega} - \mathbf{b}_g$ 이므로:

$$\frac{\partial \dot{\mathbf{q}}}{\partial \mathbf{b}_g} = -\frac{1}{2} \Xi_{1:3}(\mathbf{q}) \in \mathbb{R}^{4 \times 3} \quad (\text{B.3})$$

여기서 $\Xi_{1:3}(\mathbf{q})$ 는 $\Xi(\mathbf{q})$ 의 우측 3열로 구성된 부분 행렬:

$$\Xi_{1:3}(\mathbf{q}) = \begin{bmatrix} -q_1 & -q_2 & -q_3 \\ q_0 & -q_3 & q_2 \\ q_3 & q_0 & -q_1 \\ -q_2 & q_1 & q_0 \end{bmatrix} \quad (\text{B.4})$$

전체 $\mathbf{F}_c$

$$\mathbf{F}_c = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{1}{2} \Omega(\hat{\omega}) & -\frac{1}{2} \Xi_{1:3}(\hat{\mathbf{q}}) \\ \mathbf{0}_{3 \times 4} & \mathbf{0}_{3 \times 3} \end{bmatrix} \in \mathbb{R}^{7 \times 7} \quad (\text{B.5})$$

이산시간 상태 전이 행렬:

$$\mathbf{F}_k = \mathbf{I}_7 + \mathbf{F}_c \Delta t \quad (\text{B.6})$$

B.2 가속도계 관측 Jacobian $\mathbf{H}_a$

관측 함수 $\mathbf{h}_a(\mathbf{q}) = \mathbf{R}(\mathbf{q})\mathbf{g}^n$, $\mathbf{g}^n = [0, 0, g]^\top$ 을 전개하면:

$$\mathbf{h}_a(\mathbf{q}) = g \begin{bmatrix} 2(q_1 q_3 - q_0 q_2) \\ 2(q_2 q_3 + q_0 q_1) \\ q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix} \quad (\text{B.7})$$

각 성분을 q_0, q_1, q_2, q_3 에 대해 편미분하면:

$$\frac{\partial h_{a,1}}{\partial \mathbf{q}} = 2g \begin{bmatrix} -q_2 & q_3 & -q_0 & q_1 \end{bmatrix} \quad (\text{B.8})$$

$$\frac{\partial h_{a,2}}{\partial \mathbf{q}} = 2g \begin{bmatrix} q_1 & q_0 & q_3 & q_2 \end{bmatrix} \quad (\text{B.9})$$

$$\frac{\partial h_{a,3}}{\partial \mathbf{q}} = 2g \begin{bmatrix} q_0 & -q_1 & -q_2 & q_3 \end{bmatrix} \quad (\text{B.10})$$

따라서:

$$\frac{\partial \mathbf{h}_a}{\partial \mathbf{q}} = 2g \begin{bmatrix} -q_2 & q_3 & -q_0 & q_1 \\ q_1 & q_0 & q_3 & q_2 \\ q_0 & -q_1 & -q_2 & q_3 \end{bmatrix} \in \mathbb{R}^{3 \times 4} \quad (\text{B.11})$$

바이어스에 대한 편미분은 $\mathbf{0}_{3 \times 3}$ 이므로 전체 관측 Jacobian:

$$\mathbf{H}_a = \left[\frac{\partial \mathbf{h}_a}{\partial \mathbf{q}} \mid \mathbf{0}_{3 \times 3} \right] \in \mathbb{R}^{3 \times 7} \quad (\text{B.12})$$

B.3 지자기계 관측 Jacobian $\mathbf{H}_m$

관측 함수 $\mathbf{h}_m(\mathbf{q}) = \mathbf{R}(\mathbf{q})\mathbf{m}^n$, $\mathbf{m}^n = [m_N, 0, m_D]^\top$ 을 전개하면:

$$\mathbf{h}_m(\mathbf{q}) = \begin{bmatrix} m_N(1 - 2q_2^2 - 2q_3^2) + 2m_D(q_1 q_3 - q_0 q_2) \\ 2m_N(q_1 q_2 + q_0 q_3) + 2m_D(q_2 q_3 - q_0 q_1) \\ 2m_N(q_1 q_3 - q_0 q_2) + m_D(1 - 2q_1^2 - 2q_2^2) \end{bmatrix} \quad (\text{B.13})$$

각 성분을 q_0, q_1, q_2, q_3 에 대해 편미분하면:

$$\frac{\partial \mathbf{h}_m}{\partial \mathbf{q}} = 2 \begin{bmatrix} -m_D q_2 & m_D q_3 & -2m_N q_2 - m_D q_0 & -2m_N q_3 + m_D q_1 \\ m_N q_3 - m_D q_1 & m_N q_2 + m_D q_0 & m_N q_1 - m_D q_3 & m_N q_0 + m_D q_2 \\ -m_N q_2 & m_N q_3 - 2m_D q_1 & -m_N q_0 - 2m_D q_2 & m_N q_1 \end{bmatrix} \quad (\text{B.14})$$

전체 관측 Jacobian:

$$\mathbf{H}_m = \left[\frac{\partial \mathbf{h}_m}{\partial \mathbf{q}} \mid \mathbf{0}_{3 \times 3} \right] \in \mathbb{R}^{3 \times 7} \quad (\text{B.15})$$

C. Code Structure

본 부록에서는 Python으로 구현한 시뮬레이션 및 AHRS 알고리즘의 코드 구조를 설명한다.

C.1 전체 디렉토리 구조

```
prj_imu_fusion/
├── sim/
│   ├── imu_simulator.py    # IMU 데이터 합성
│   ├── trajectory.py       # 참 자세 궤적 생성
│   └── noise_model.py      # 잡음 모델 (ARW, RRW)
├── filters/
│   ├── complementary.py   # Complementary Filter
│   ├── mahony.py          # Mahony Filter
│   ├── madgwick.py        # Madgwick Filter
│   └── ekf.py             # Extended Kalman Filter
├── calibration/
│   ├── accel_calib.py     # 가속도계 캘리브레이션
│   ├── gyro_calib.py     # 자이로 Allan Variance
│   └── mag_calib.py       # Hard/Soft Iron 보정
├── analysis/
│   ├── allan_variance.py  # AVAR 계산 및 시각화
│   └── psd.py             # PSD 분석
├── hardware/
│   └── mpu9250_reader.py   # Arduino 시리얼 통신
└── main.py                # 메인 실행 스크립트
```

C.2 핵심 모듈 설명

imu_simulator.py

```
class IMUSimulator:
    def __init__(self, sigma_w, sigma_rw, sigma_a, sigma_m, fs):
        # 잡음 파라미터 초기화

    def generate(self, q_true, omega_true):
        # 자이로, 가속도계, 지자기계 측정값 합성
        # 반환: gyro_meas, accel_meas, mag_meas
```

ekf.py

```
class EKF:
    def __init__(self, Q, Ra, Rm, P0):
        # 공분산 초기화
        self.x = np.array([1,0,0,0, 0,0,0]) # 상태 벡터
        self.P = P0

    def predict(self, gyro, dt):
        # 상태 전파 및 공분산 예측
        #  $F = I + F_c * dt$ 

    def update_accel(self, accel):
        # 가속도계 Update (동적 가속도 조건 포함)
        #  $H_a, K_a$ , 상태/공분산 갱신

    def update_mag(self, mag):
        # 지자기계 Update
        #  $H_m, K_m$ , 상태/공분산 갱신

    def normalize(self):
        # 쿼터니언 정규화

    def get_euler(self):
        # 쿼터니언 → 오일러각 변환
```

allan_variance.py

```
def compute_avar(data, fs):
    # 입력: 정지 상태 자이로 시계열, 샘플링 주파수
    # 출력: tau 배열, AVAR 배열
    # 방법: overlapping AVAR

def extract_noise_params(tau, avar):
    # log-log 기울기 분석으로 ARW, BI, RRW 계수 추출
    # 반환: N (ARW), B (BI), K (RRW)
```

C.3 실행 방법**시뮬레이션 실행**

```
# 의존성 설치
pip install numpy scipy matplotlib

# 시뮬레이션 실행
python main.py --mode sim --filter ekf --duration 60

# 필터 비교
python main.py --mode sim --filter all --plot
```

실제 데이터 분석

```
# Arduino 시리얼 데이터 수집 (baudrate: 115200)
```

```
python hardware/mpu9250_reader.py --port COM3 --duration 7200

# Allan Variance 분석
python analysis/allan_variance.py --input data/gyro_static.csv

# 캘리브레이션 실행
python calibration/accel_calib.py --input data/accel_6pos.csv
python calibration/mag_calib.py --input data/mag_rotation.csv
```

D. MPU-9250 Hardware Overview

본 부록에서는 MPU-9250 IMU의 하드웨어 사양과 통신 인터페이스를 정리한다.

D.1 센서 사양

Table 5: MPU-9250 센서 사양

항목	가속도계	자이로스코프	지자기계 (AK8963)
측정 범위	$\pm 2/4/8/16$ g	$\pm 250/500/1000/2000$ °/s	± 4800 μ T
ADC 해상도	16 bit	16 bit	14 / 16 bit
감도 (기본)	16384 LSB/g	131 LSB/(°/s)	0.6 μ T/LSB
출력 데이터 레이트	최대 4 kHz	최대 8 kHz	최대 100 Hz
동작 전압		1.71 – 3.45 V	
소비 전류	450 μ A	3.2 mA	280 μ A
온도 범위		$-40^{\circ}\text{C} \sim +85^{\circ}\text{C}$	
패키지		QFN $3 \times 3 \times 1$ mm	

D.2 I²C 인터페이스

MPU-9250은 호스트와 I²C 프로토콜로 통신하며, AD0 핀 레벨에 따라 슬레이브 주소가 결정된다:

$$\text{AD0} = \text{LOW} \Rightarrow 0x68, \quad \text{AD0} = \text{HIGH} \Rightarrow 0x69 \quad (\text{D.1})$$

주요 레지스터:

Table 6: MPU-9250 주요 레지스터

레지스터 주소	이름	설명
0x3B – 0x40	ACCEL_XOUT – ACCEL_ZOUT	가속도계 원시 데이터 (16비트 $\times$ 3축)
0x41 – 0x42	TEMP_OUT	온도 데이터
0x43 – 0x48	GYRO_XOUT – GYRO_ZOUT	자이로 원시 데이터 (16비트 $\times$ 3축)
0x6B	PWR_MGMT_1	전원 관리 및 클럭 설정
0x1B	GYRO_CONFIG	자이로 측정 범위 설정
0x1C	ACCEL_CONFIG	가속도계 측정 범위 설정
0x37	INT_PIN_CFG	Pass-Through 모드 활성화

원시 데이터 변환:

$$\text{raw} = (\text{HIGH_BYTE} \ll 8) | \text{LOW_BYTE} \quad (\text{D.2})$$

$$\text{physical value} = \frac{\text{raw}}{\text{sensitivity}} \quad (\text{D.3})$$

D.3 AK8963 지자기계 접근

AK8963은 MPU-9250의 보조 I²C 버스에 연결되어 있으며, Pass-Through 모드를 활성화하면 호스트에서 직접 접근할 수 있다.

1. MPU-9250 레지스터 0x6A (USER_CTRL)의 I2C_MST_EN 비트를 0으로 설정
2. MPU-9250 레지스터 0x37 (INT_PIN_CFG)의 BYPASS_EN 비트를 1로 설정
3. AK8963 슬레이브 주소 0x0C로 직접 접근
4. AK8963 레지스터 0x0A (CNTL1)에 0x16 기록: 16비트 연속 측정 모드 2

AK8963 주요 레지스터:

Table 7: AK8963 주요 레지스터

레지스터 주소	이름	설명
0x02	ST1	데이터 준비 상태 (DRDY 비트)
0x03 – 0x08	HXL – HZH	자기장 원시 데이터 (16비트 × 3축)
0x09	ST2	데이터 오버런 및 자기 센서 오버플로
0x0A	CNTL1	측정 모드 및 출력 비트 설정
0x10 – 0x12	ASAX/Y/Z	감도 보정값 (ROM)

감도 보정값(Sensitivity Adjustment Value) 적용:

$$H_{\text{adj},i} = H_i \times \frac{(ASA_i - 128) \times 0.5}{128} + H_i, \quad i \in \{x, y, z\} \quad (\text{D.4})$$

D.4 좌표계 정렬

MPU-9250 가속도계/자이로스코프와 AK8963 지자기계는 서로 다른 좌표계를 사용하므로 데이터 퓨전 전 축 변환이 필요하다.

Table 8: MPU-9250 좌표계 정렬

센서	X축	Y축	Z축
가속도계 / 자이로스코프	전방	우측	하방
AK8963 지자기계	우측	전방	하방
정렬 후 (Body Frame)	전방	우측	하방

AK8963 출력의 Body Frame 변환:

$$\begin{bmatrix} m_x^b \\ m_y^b \\ m_z^b \end{bmatrix} = \begin{bmatrix} m_y^{\text{AK}} \\ m_x^{\text{AK}} \\ -m_z^{\text{AK}} \end{bmatrix} \quad (\text{D.5})$$

References

- [1] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.
- [2] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [3] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [4] Jean Kossaifi, Yannis Panagakis, Anima Anandkumar, and Maja Pantic. TensorLy: Tensor learning in Python. *Journal of Machine Learning Research*, 20(26):1–6, 2019.